

ECE 154B Lab report 2

Branch Predictor

ECE 154B Computer Architecture

Team Member: Zheng Zhang, Hang Ye, NingJing Zhang

Date: 2/18/2024

Introduction:

In contemporary processor design, branch prediction technology stands as a pivotal factor in enhancing processor performance. Accurate branch prediction can significantly reduce delays caused by control flow changes encountered during instruction execution, particularly in processors with deep pipelines and high levels of Instruction-Level Parallelism (ILP). The combined use of Branch Target Buffer (BTB), Pattern History Table (PHT), and Global History Register (GHR) forms a complex and powerful branch prediction mechanism, capable of improving prediction accuracy and overall processor performance.

BTB (Branch Target Buffer)

The BTB stores target addresses of recently accessed branch instructions, enabling the processor to quickly obtain the branch target addresses without waiting for the branch instruction to be fully executed. This allows the processor to fetch and execute instructions at the branch target ahead of time, reducing delays caused by waiting for branch resolution.

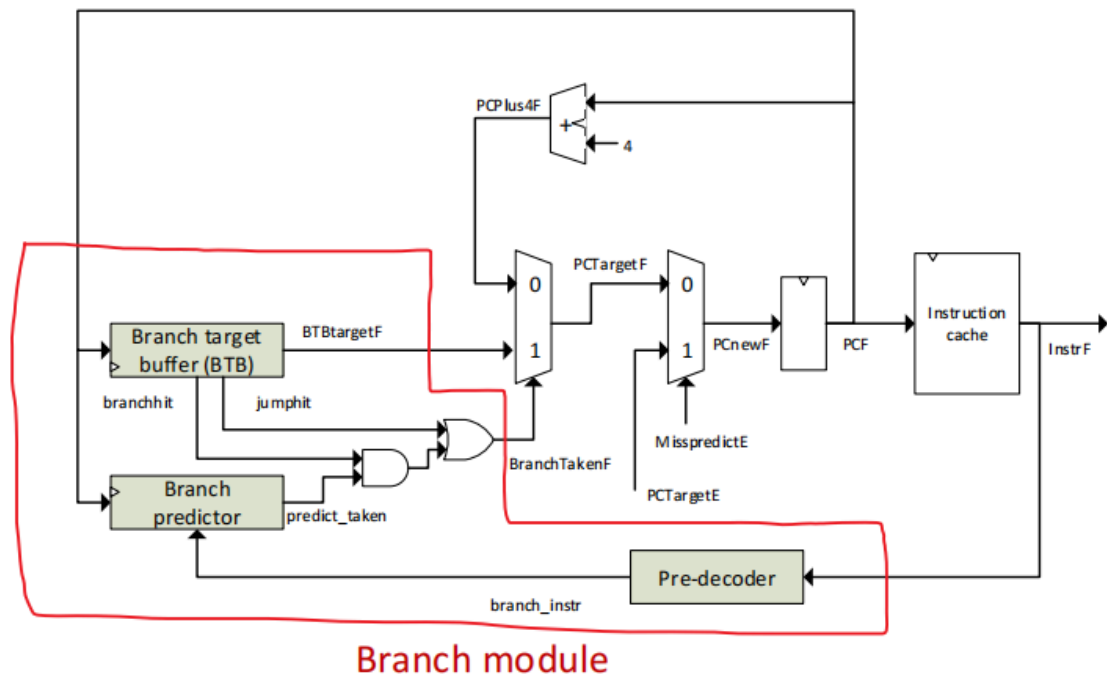
PHT (Pattern History Table)

The PHT stores pattern information of branch history, typically working in conjunction with the GHR to provide history-based branch prediction. Each time a branch is executed, its outcome (taken or not taken) is recorded in the PHT based on the state of the GHR. This information is subsequently used to predict the behavior of future branches under the same historical pattern.

GHR (Global History Register)

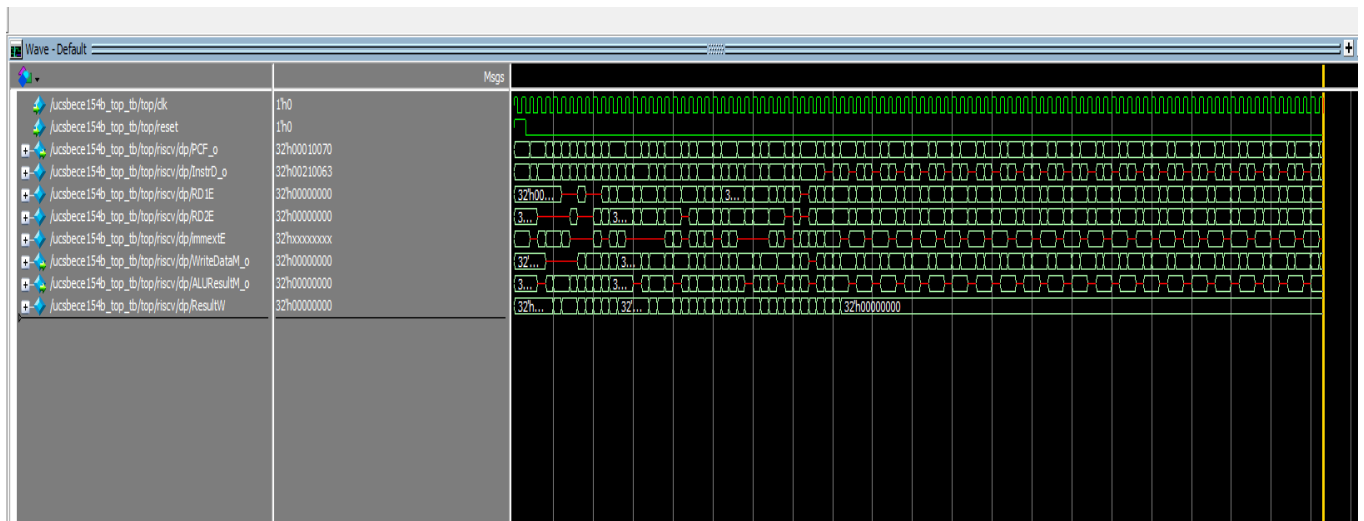
The GHR records the taken/not-taken status of recently executed branch instructions, providing a global view of branch history. This historical information is used to index the PHT, enabling branch prediction to consider patterns in past branch behavior.

The branch predictor that utilizes BTB, PHT, and GHR in combination leverages the advantages of these three mechanisms to achieve higher prediction accuracy. The GHR provides a global branch history, allowing the PHT to make predictions based on past branch patterns, while the BTB ensures rapid retrieval of branch target addresses. This prediction mechanism not only considers the local history of branches but also global branch patterns, thus enabling more accurate prediction of branch behavior in programs with complex control flows.

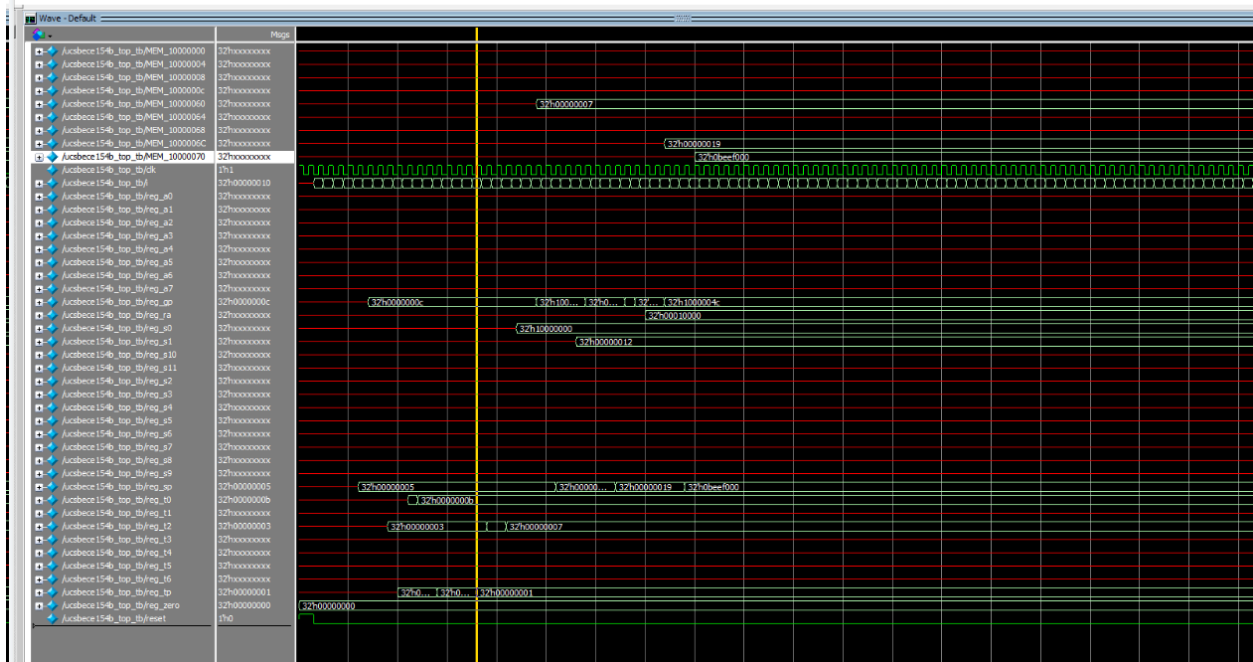


Result:

Here is the whole waveform which including :CLK, Reset, PCF, InstD, RD1E, RD2E, ExtImmE, ALUResultM, WriteDataM, ResultW signals:



In additional, Here is the verification register waveform:



Compare with the test case:

```
`ASSERT(reg_zero==32'b0, ("reg_zero incorrect"));
`ASSERT(reg_sp==32'hBEEF000, ("reg_sp incorrect"));
`ASSERT(reg_gp==32'h1000004C, ("reg_gp incorrect"));
`ASSERT(reg_tp==32'h1, ("reg_tp incorrect"));
`ASSERT(reg_t0==32'hb, ("reg_t0 incorrect"));
`ASSERT(reg_t2==32'h7, ("reg_t2 incorrect"));
`ASSERT(MEM_10000060==32'h7, ("mem.DATA[25] incorrect"));
`ASSERT(MEM_1000006C==32'h19, ("mem.DATA[28] incorrect"));
`ASSERT(MEM_10000070==32'hBEEF000, ("mem.DATA[29] //incorrect"));
```

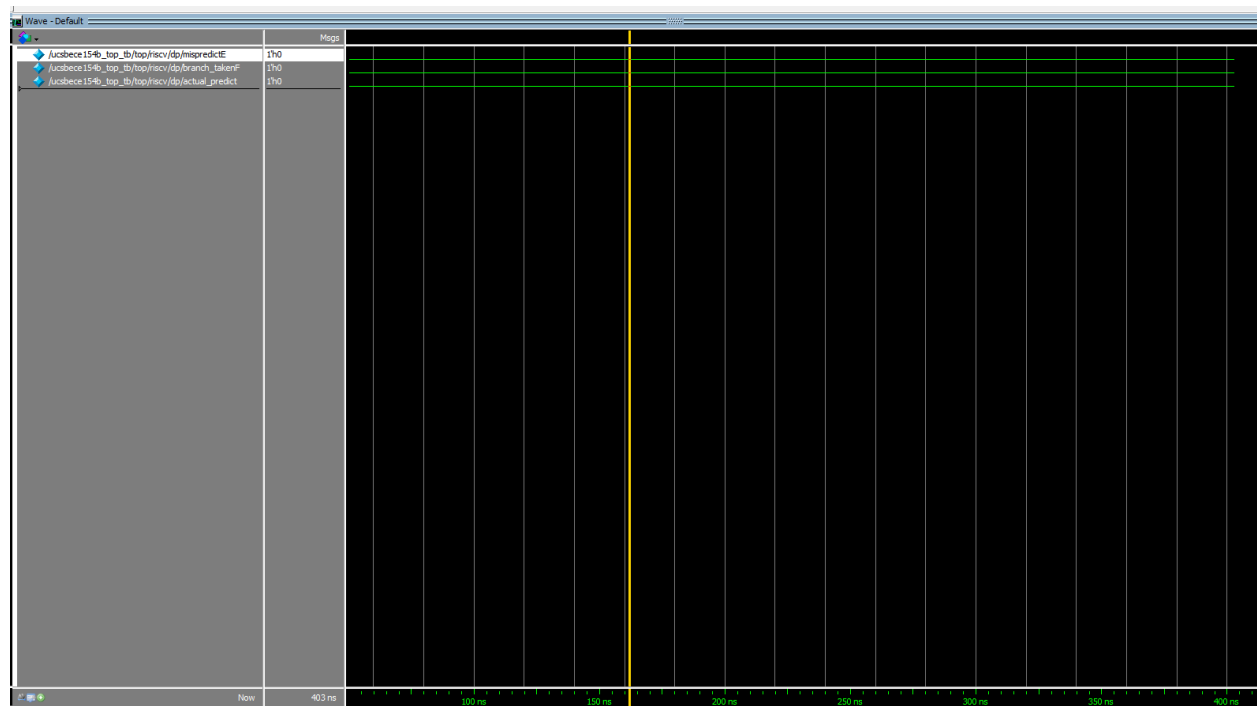
We can find that both zero,sp,gp,tp,t0,t2,Mem_10000060,Mem_1000006C,Mem_10000070 Have the same value with the above waveform, And local transcript also indicates the code pass the test:

```

3 Transcript
# Warning: (vrain-WLF-9000) WLF file currently in use: vrain.wlf
# File in use by: zheng Hostname: 224 ProcessID: 19272
# Attempting to use alternate WLF file: "vraic13007"
# ** Warning: (vrain-WLF-9001) Could not open WLF file: vrain.wlf
# Using alternative file: ./vraic13007
VDM 000> restart -f
# ** Warning: (vrain-0001) All optimizations are turned off because the -noopt switch is in effect. This will cause your simulation to run very slowly. If you are using this switch to preserve visibility for Debug or FLI features please see the User's Manual section on Preserving Object Visibility with vopt.
VDM 000> run -all
# Begin simulation.
# End simulation.
# ** Note: setup : C:/Users/zheng/Desktop/1404b/real1404 lab1 v2/riscvstarter/ucbcece1404_top_tb.v(96)
# Time: 203 ns Iteration: 1 Database: /ucbcece1404_top_tb
# Break in Module ucbcece1404_top_tb at C:/Users/zheng/Desktop/1404b/real1404 lab1 v2/riscvstarter/ucbcece1404_top_tb.v line 96
VDM 000> quit -sim
# End time: 19.11.20 on Jan 23, 2024, Elapsed time: 0103119
# Error: 0, Warnings: 4
ModelSim>

```

For Branch predictor,



There is still some problem with our code, the Branch predictor does not work well, we know we have one more chance to revise the design file so we will try to fix it.

Q&A:

. Limits / Cons for Increasing the Length of GHR and BTB

Global History Register (GHR): Increasing the length of the GHR allows the branch predictor to consider a longer history of branch outcomes, potentially improving prediction accuracy for patterns that span many branches. However, there are several drawbacks:

Increased Complexity: A longer GHR means more bits to manage and more complex hardware to interpret the history.

Diminishing Returns: As the GHR length increases, the improvement in prediction accuracy may plateau or even decrease due to the aliasing of branch histories.

Increased Power Consumption: Managing a longer GHR requires more power, affecting the energy efficiency of the processor.

Branch Target Buffer (BTB): The BTB stores the target addresses of recently taken branches to speed up the execution of branch instructions. Increasing its size can:

Improve Hit Rate: A larger BTB can store more branch target addresses, potentially reducing the number of costly misses.

Cons:

Increased Cache Pressure: A larger BTB can consume more cache space, potentially reducing the effectiveness of other caches.

Higher Latency and Power Consumption: Accessing a larger BTB can take more time and consume more power.

ii. Why Speculatively Update GHR?

Speculatively updating the GHR means updating the branch history with the predicted outcome of branches before they are actually resolved. This approach is used to:

Reduce Latency: By speculatively updating the GHR, the processor can make branch predictions for subsequent instructions without waiting for the current branch to be resolved, reducing the pipeline stall time.

Example: Consider a loop with a conditional branch at the end that depends on the loop counter. Speculatively updating the GHR allows the processor to predict the outcome of the branch in the next iteration of the loop before the current iteration completes, thereby keeping the pipeline filled.

iii. Is Improving Branch Prediction Beyond Implemented Scheme Critical for the Considered 5-stage Processor?

The necessity of improving branch prediction in a 5-stage pipeline processor depends on the architecture's specific characteristics and the types of workloads it runs. If the processor experiences a high number of branch mispredictions that significantly impact performance, enhancing the branch predictor could yield substantial benefits. However, for simple processors or workloads with easily predictable branches, the existing scheme might be sufficient.

iv. What Processors Would Benefit from Better Branch Predictors?

High-Performance Processors: CPUs with deep pipelines and high instruction-level parallelism (ILP) can benefit significantly from improved branch prediction to maintain pipeline efficiency and minimize stalls.

Processors Running Complex Workloads: Workloads with complex control flow, such as database systems, large-scale simulations, and software development tools, would benefit from better branch predictors to improve overall performance.

v. How to Minimize the Delay of Pre-Decoding?

Minimizing the delay of pre-decoding can be achieved by:

Streamlining the Pre-Decoding Process: Simplify the pre-decoding logic to reduce the number of cycles needed to pre-decode an instruction.

Parallel Processing: Use parallel pre-decoders to handle multiple instructions simultaneously, reducing the overall pre-decoding time.

Instruction Cache Optimization: Optimize the instruction cache layout to speed up instruction fetch and pre-decoding.

vi. More Efficient Scheme for Resetting GSR in Case of Branch Misprediction

Selective Reset: Instead of resetting the entire Global Shift Register (GSR) on a misprediction, selectively reset or roll back only the bits affected by the mispredicted branch. This approach minimizes the disruption to branch prediction accuracy.

Checkpointing: Implement a checkpointing system where the state of the GSR is saved at certain points. Upon a misprediction, the GSR can be quickly restored to the last checkpoint, reducing the time and complexity of the reset process.

Working time:

Since all three members do not take the 154A, we spend more time on it. And for this lab we also do not have gradescop to help identify the signal should be. It is a huge difficulty for us, we spend between 50-100 hours on this lab.